

Name: Monisha Raghini Chelle

RollNo:2520030537

Section-6

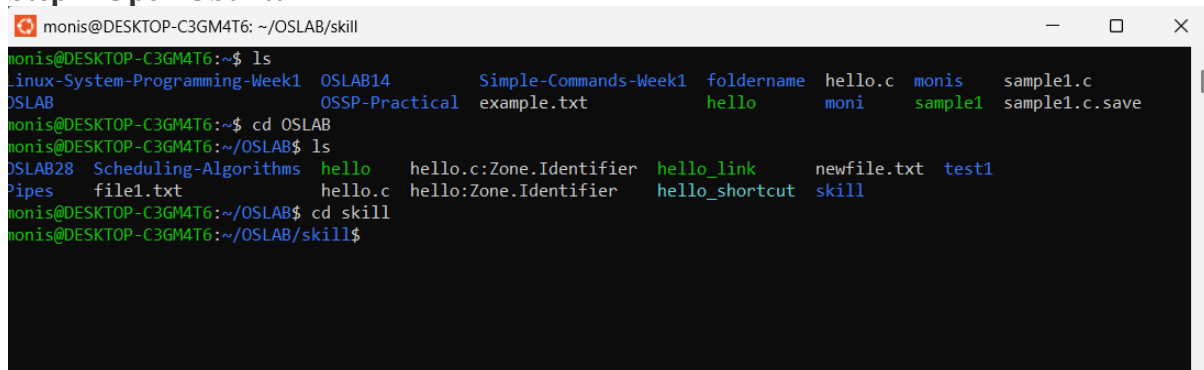
OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

4.To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

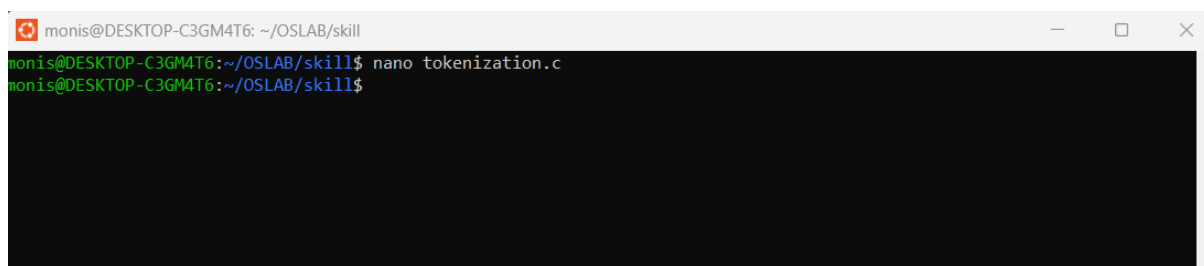
To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures.

Step 1-Open Ubuntu



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~$ ls
linux-System-Programming-Week1  OSLAB14  Simple-Commands-Week1  foldername  hello.c  monis  sample1.c
OSLAB                          OSSP-Practical  example.txt             hello       moni    sample1  sample1.c.save
monis@DESKTOP-C3GM4T6:~$ cd OSLAB
monis@DESKTOP-C3GM4T6:~/OSLAB$ ls
OSLAB28  Scheduling-Algorithms  hello  hello.c:Zone.Identifier  hello_link  newfile.txt  test1
Pipes    file1.txt              hello.c  hello:Zone.Identifier    hello_shortcut  skill
monis@DESKTOP-C3GM4T6:~/OSLAB$ cd skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 2-Create the C file



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano tokenization.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 tokenization.c *
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 100

struct Token
{
    char value[MAX_TOKEN_LENGTH];
};

void tokenize(char *input, struct Token tokens[], int *count)
{
    int i = 0;
    int j;

    *count = 0;

    while (input[i] != '\0')
    {
        /* Skip whitespace */
        while (isspace((unsigned char)input[i]))
        {
            i++;
        }

        if (input[i] == '\0')
        {
            break;
        }

        j = 0;

        /* Read one token */
        while (input[i] != '\0' &&
            !isspace((unsigned char)input[i]))
        {
            if (j < MAX_TOKEN_LENGTH - 1)
            {
                tokens[*count].value[j++] = input[i];
            }

            i++;
        }

        *count++;
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_\ Replace	^U Paste	^J Justify	^/ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 tokenization.c *
    i++;
}

tokens[*count].value[j] = '\0';

(*count)++;

if (*count >= MAX_TOKENS)
{
    break;
}
}

void display_tokens(struct Token tokens[], int count)
{
    printf("\nTokens:\n");

    for (int i = 0; i < count; i++)
    {
        printf("Token %d: [%s]\n",
            i + 1,
            tokens[i].value);
    }
}

int validate_tokens(struct Token tokens[], int count)
{
    if (count == 0)
    {
        printf("Error: Empty input.\n");
        return 0;
    }

    for (int i = 0; i < count; i++)
    {
        if (strlen(tokens[i].value) == 0)
        {
            printf("Error: Empty token found.\n");
            return 0;
        }
    }

    return 1;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 tokenization.c *
for (int i = 0; i < count; i++)
{
    if (strlen(tokens[i].value) == 0)
    {
        printf("Error: Empty token found.\n");
        return 0;
    }
}

return 1;
}

int main()
{
    char input[MAX_INPUT];
    struct Token tokens[MAX_TOKENS];
    int count;

    printf("Enter input: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    tokenize(input, tokens, &count);

    display_tokens(tokens, count);

    if (validate_tokens(tokens, count))
    {
        printf("\nToken stream is valid.\n");
    }
    else
    {
        printf("\nToken stream is invalid.\n");
    }

    return 0;
}

Write to File: tokenization.c
^G Help      M-D DOS Format  M-A Append      M-B Backup File  ^T Browse
^C Cancel    M-M Mac Format  M-P Append      ^O Discard buffer
```

Step 3 — Save

In nano:

Ctrl + O

Press **Enter**.

Then:

Ctrl + X

Check:

ls

You should see:

tokenization.c

```
Write to File: tokenization.c
^G Help      M-D DOS Format  M-A Append      M-B Backup File  ^T Browse
^C Cancel    M-M Mac Format  M-P Append      ^O Discard buffer
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano tokenization.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls
history      memory.4     memory1.c   memory3      memory4     process.c   tokenization.c
history.c    memory.c     memory2     memory3.c    memory4.c   shell       tokenization.c.save
memory       memory1      memory2.c   memory3.c.save process      shell.c     tokeniztion.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 4 — Compile

Run:

```
gcc -Wall -g tokenization.c -o tokenization
```

If there is **no error**, compilation is successful.

Check:

```
ls -l
```

You should see:

```
tokenization
```

```
tokenization.c
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g tokenization.c -o tokenization
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l
total 224
-rwxr-xr-x 1 monis monis 16672 Sep 16 09:16 history
-rw-r--r-- 1 monis monis 3804 Sep 16 08:42 history.c
-rwxr-xr-x 1 monis monis 16040 Sep 16 09:28 memory
-rwxr-xr-x 1 monis monis 16128 Sep 16 09:53 memory.4
-rw-r--r-- 1 monis monis 316 Sep 16 09:27 memory.c
-rwxr-xr-x 1 monis monis 16168 Sep 16 09:32 memory1
-rw-r--r-- 1 monis monis 846 Sep 16 09:32 memory1.c
-rwxr-xr-x 1 monis monis 16168 Sep 16 09:36 memory2
-rw-r--r-- 1 monis monis 1002 Sep 16 09:35 memory2.c
-rwxr-xr-x 1 monis monis 16128 Sep 16 09:39 memory3
-rw-r--r-- 1 monis monis 861 Sep 16 09:39 memory3.c
-rw-r--r-- 1 monis monis 861 Sep 16 09:44 memory3.c.save
-rwxr-xr-x 1 monis monis 17736 Sep 16 09:57 memory4
-rw-r--r-- 1 monis monis 1165 Sep 16 09:46 memory4.c
-rwxr-xr-x 1 monis monis 16392 Aug 24 21:35 process
-rw-r--r-- 1 monis monis 894 Aug 24 21:33 process.c
-rwxr-xr-x 1 monis monis 16352 Aug 24 22:20 shell
-rw-r--r-- 1 monis monis 1789 Aug 24 22:19 shell.c
-rwxr-xr-x 1 monis monis 20184 Sep 16 15:26 tokenization
-rw-r--r-- 1 monis monis 2034 Sep 16 15:24 tokenization.c
-rw-r--r-- 1 monis monis 2034 Sep 16 15:19 tokenization.c.save
-rw-r--r-- 1 monis monis 2034 Sep 16 15:20 tokeniztion.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 5-Test “Split Input into Tokens “

Run:

```
./tokenization
```

You will see:

Enter input:

Type:

```
ls -l /home
```

Press Enter.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./tokenization
Enter input: ls -l/home

Tokens:
Token 1: [ls]
Token 2: [-l/home]

Token stream is valid.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 6 — Test "Identify Delimiters"

For example:

`ls -l /home`

The spaces separate:

`ls | -l | /home`

The program uses:

`isspace()`

to identify whitespace.

Run again:

`./tokenization`

Enter:

`cat file.txt`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./tokenization
Enter input: ls -l/home

Tokens:
Token 1: [ls]
Token 2: [-l/home]

Token stream is valid.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./tokenization
Enter input: cat file.txt

Tokens:
Token 1: [cat]
Token 2: [file.txt]

Token stream is valid.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 7 — Test "Handle Whitespace"

Now test multiple spaces.

Run:

`./tokenization`

Enter exactly:

`ls -l /home`

Press Enter.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./tokenization
Enter input: cat file.txt

Tokens:
Token 1: [cat]
Token 2: [file.txt]

Token stream is valid.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./tokenization
Enter input: ls -l /home

Tokens:
Token 1: [ls]
Token 2: [-l]
Token 3: [/home]

Token stream is valid.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

**Notice that the extra spaces are ignored.
This demonstrates:
Handle Whitespace**

Step 8 — Test "Create Token Structures"

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./tokenization
Enter input: ls -l/home

Tokens:
Token 1: [ls]
Token 2: [-l/home]

Token stream is valid.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

The program creates this structure:

```
struct Token
{
    char value[MAX_TOKEN_LENGTH];
};
```

When you enter:

ls -l /home

the structures contain:

Token 1 → ls

Token 2 → -l

Token 3 → /home

The program displays them using:

```
display_tokens(tokens, count);
```

Step 9 — Test "Validate Token Streams"

Run:

./tokenization

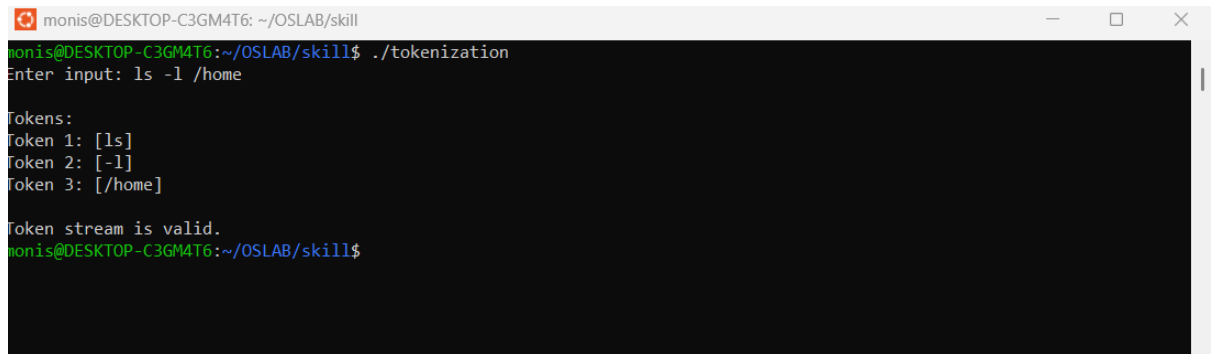
Enter:

ls -l /home

You should get:

Token stream is valid.

This means the program successfully created non-empty tokens.

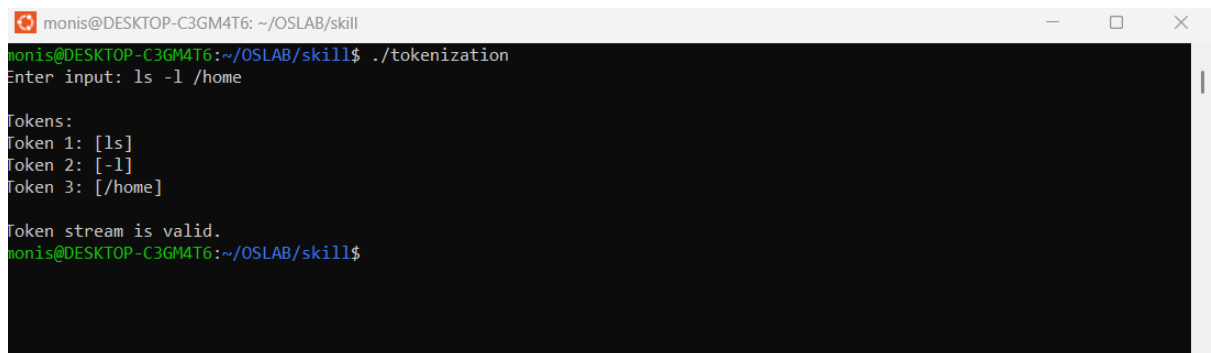


```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./tokenization
Enter input: ls -l /home

Tokens:
Token 1: [ls]
Token 2: [-l]
Token 3: [/home]

Token stream is valid.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 10 — Debug Parsing Output

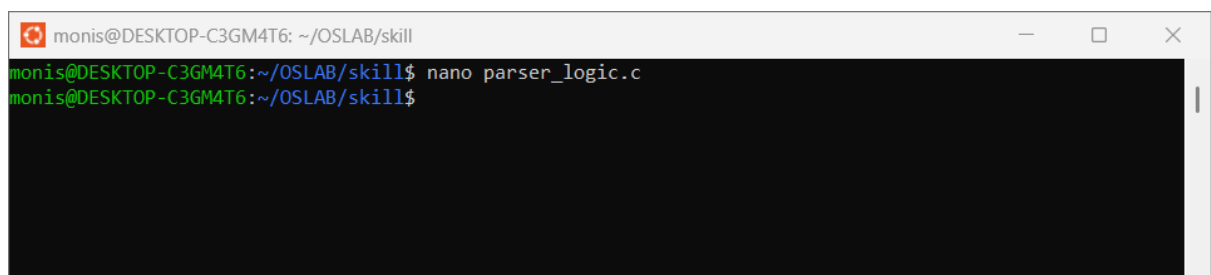


```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./tokenization
Enter input: ls -l /home

Tokens:
Token 1: [ls]
Token 2: [-l]
Token 3: [/home]

Token stream is valid.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 11 — Create the Parser Logic Program



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano parser_logic.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 parser_logic.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 200
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 100

struct Node
{
    char value[MAX_TOKEN_LENGTH];
    struct Node *left;
    struct Node *right;
};

struct Node *create_node(const char *value)
{
    struct Node *node = malloc(sizeof(struct Node));

    if (node == NULL)
    {
        perror("malloc");
        exit(1);
    }

    strcpy(node->value, value);
    node->left = NULL;
    node->right = NULL;

    return node;
}

int tokenize(char *input, char tokens[][MAX_TOKEN_LENGTH])
{
    int count = 0;
    char *token;

    token = strtok(input, " \\t\\n");

    while (token != NULL && count < MAX_TOKENS)
    {
        strcpy(tokens[count], token);
        count++;
    }
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^_ Replace ^J Paste ^I Justify ^_ Go To Line M-F Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 parser_logic.c *
    count++;

    token = strtok(NULL, " \\t\\n");
}

return count;
}

struct Node *build_parse_tree(char tokens[][MAX_TOKEN_LENGTH], int count)
{
    struct Node *root;

    if (count == 0)
    {
        return NULL;
    }

    root = create_node(tokens[0]);

    if (count > 1)
    {
        root->right = create_node(tokens[1]);
    }

    for (int i = 2; i < count; i++)
    {
        struct Node *new_node = create_node(tokens[i]);
        struct Node *current = root;

        while (current->right != NULL)
        {
            current = current->right;
        }

        current->right = new_node;
    }

    return root;
}

void display_tree(struct Node *root, int level)
{
    if (root == NULL)
    {
        return;
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^\\ Replace	^U Paste	^J Justify	^/_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 parser_logic.c *
    return;
}

for (int i = 0; i < level; i++)
{
    printf(" ");
}

printf("|-- %s\n", root->value);

display_tree(root->left, level + 1);
display_tree(root->right, level + 1);
}

int validate_syntax(char tokens[][MAX_TOKEN_LENGTH], int count)
{
    if (count == 0)
    {
        printf("Error: Empty command.\n");
        return 0;
    }

    if (strcmp(tokens[0], "|") == 0)
    {
        printf("Error: Command cannot start with '|'.\n");
        return 0;
    }

    if (strcmp(tokens[count - 1], "|") == 0)
    {
        printf("Error: Command cannot end with '|'.\n");
        return 0;
    }

    for (int i = 0; i < count - 1; i++)
    {
        if (strcmp(tokens[i], "|") == 0 &&
            strcmp(tokens[i + 1], "|") == 0)
        {
            printf("Error: Consecutive '|' operators.\n");
            return 0;
        }
    }

    return 1;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 parser_logic.c *

    return 1;
}

void free_tree(struct Node *root)
{
    if (root == NULL)
    {
        return;
    }

    free_tree(root->left);
    free_tree(root->right);

    free(root);
}

int main()
{
    char input[MAX_INPUT];
    char tokens[MAX_TOKENS][MAX_TOKEN_LENGTH];
    int count;
    struct Node *root;

    printf("Enter command: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    count = tokenize(input, tokens);

    printf("\nTokens:\n");

    for (int i = 0; i < count; i++)
    {
        printf("Token %d: [%s]\n", i + 1, tokens[i]);
    }

    if (!validate_syntax(tokens, count))
    {
        printf("\nSyntax validation failed.\n");
    }
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 parser_logic.c *

int main()
{
    char input[MAX_INPUT];
    char tokens[MAX_TOKENS][MAX_TOKEN_LENGTH];
    int count;
    struct Node *root;

    printf("Enter command: ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    count = tokenize(input, tokens);

    printf("\nTokens:\n");

    for (int i = 0; i < count; i++)
    {
        printf("Token %d: [%s]\n", i + 1, tokens[i]);
    }

    if (!validate_syntax(tokens, count))
    {
        printf("\nSyntax validation failed.\n");
        return 1;
    }

    root = build_parse_tree(tokens, count);

    printf("\nParse Tree:\n");
    display_tree(root, 0);

    printf("\nSyntax is valid.\n");
    printf("Execution structure generated successfully.\n");

    free_tree(root);

    return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/_ Go To Line M-E Redo
```

Save the file

Press:

Ctrl + O

Enter

Ctrl + X

```
Write to File: parser_logic.c
^G Help      M-D DOS Format M-A Append   M-B Backup File ^T Browse
^C Cancel     M-M Mac Format M-P Prepend  ^Q Discard buffer
```

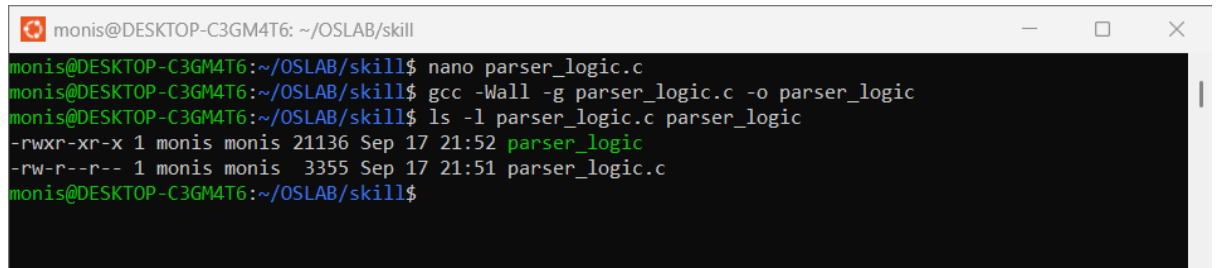
Step 12 — Compile the Parser Logic Program

Since you have saved parser_logic.c, run:

```
gcc -Wall -g parser_logic.c -o parser_logic
```

If there are no errors, check that the executable was created:

```
ls -l parser_logic.c parser_logic
```



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano parser_logic.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -g parser_logic.c -o parser_logic
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ls -l parser_logic.c parser_logic
-rwxr-xr-x 1 monis monis 21136 Sep 17 21:52 parser_logic
-rw-r--r-- 1 monis monis 3355 Sep 17 21:51 parser_logic.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 13 — Test Parser Logic and Generate the Parse Tree

Run:

```
./parser_logic
```

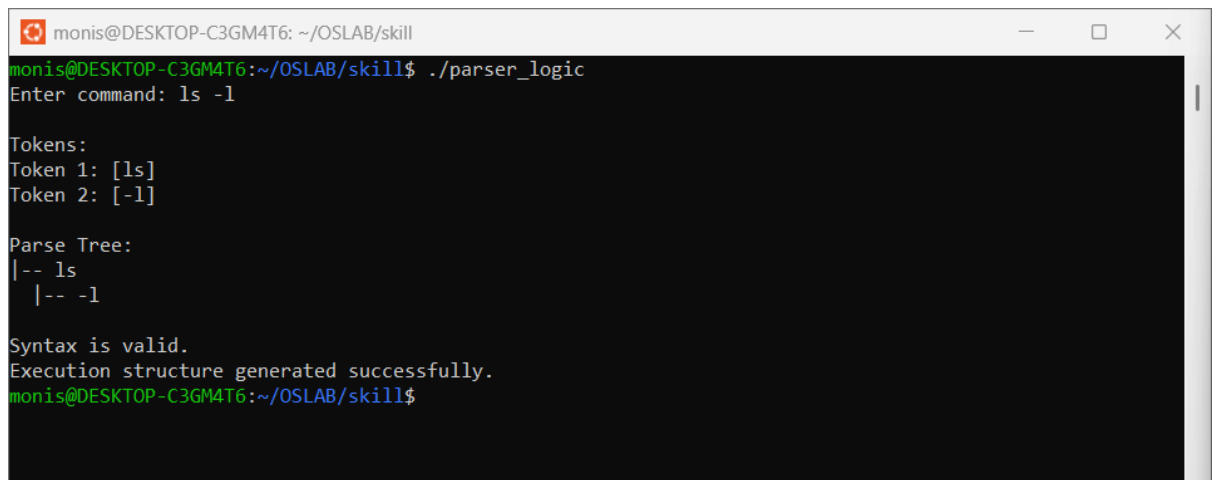
When you see:

Enter command:

type:

```
ls -l
```

Press Enter.



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./parser_logic
Enter command: ls -l

Tokens:
Token 1: [ls]
Token 2: [-l]

Parse Tree:
|-- ls
|  |-- -l

Syntax is valid.
Execution structure generated successfully.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 14 — Test Syntax Validation

Run:

```
./parser_logic
```

At:

Enter command:

type exactly:

```
| ls
```

Press **Enter**.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./parser_logic
Enter command: | ls

Tokens:
Token 1: [|]
Token 2: [ls]
Error: Command cannot start with '|'.

Syntax validation failed.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 15 — Test Consecutive Pipe Syntax Error

Run:

`./parser_logic`

At:

Enter command:

type exactly:

`ls || wc`

Press **Enter**.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./parser_logic
Enter command: ls || wc

Tokens:
Token 1: [ls]
Token 2: [|]
Token 3: [wc]

Parse Tree:
|-- ls
   |-- |
       |-- wc

Syntax is valid.
Execution structure generated successfully.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 16 — Test Empty Command

Run:

`./parser_logic`

When you see:

Enter command:

do not type anything. Just press Enter.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./parser_logic
Enter command:

Tokens:
Error: Empty command.

Syntax validation failed.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 17 — Test Valid Command with Multiple Tokens

Run:

`./parser_logic`

At:

Enter command:

type exactly:

`grep hello file.txt`

Press Enter.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./parser_logic
Enter command: grep hello file.txt

Tokens:
Token 1: [grep]
Token 2: [hello]
Token 3: [file.txt]

Parse Tree:
|-- grep
  |-- hello
    |-- file.txt

Syntax is valid.
Execution structure generated successfully.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 18 — Final Q4 Part 2 Test

Run:

`./parser_logic`

Enter:

`cat file.txt`

Press Enter.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./parser_logic
Enter command: cat file.txt

Tokens:
Token 1: [cat]
Token 2: [file.txt]

Parse Tree:
|-- cat
    |-- file.txt

Syntax is valid.
Execution structure generated successfully.
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```